SectraTEK

AGENTIC AI FOR DIGITAL FORENSICS · LAB 03

Reason, Act, Repeat



The ReAct Pattern: the model says why it needs each piece of evidence, calls one tool, reads the result, and only then decides the next step.

Daniel Kwaku Ntiamoah Addai · Mobile Threat Research Lead, SectraTEK Lab · AI for Cybersecurity and Digital Forensics

YOUR INSTRUCTOR

Daniel Kwaku Ntiamoah Addai



Mobile Threat Research Lead

SectraTEK Lab

lab.sectratek.com

IN ONE LINE

I study how phones get spied on and tampered with, and build AI tools that help examiners find the evidence.

Mobile threat research
Spyware, stalkerware and device tampering.

AI for investigators
Machine learning that speeds up the search.

Teaching
AI for Cybersecurity and Digital Forensics.

TODAY'S STORY

Reasoning you can read

1

What ReAct adds

The model shows its reasoning between tool calls.

2

Before AI

An examiner works a timeline one check at a time.

3

The problem

A model jumps from an attempt to a conclusion.

4

The discipline

One step at a time, each driven by the last result.

5

The gap

A one-shot answer hides the reasoning you must audit.

6

ReAct

Thought, action, observation, repeat, then answer.

7

The case

A phone left alone, and a Signal attempt.

8

Your lab

Walk the loop by hand, then let the agent run it.

Practical runs in Google Colab with a free hosted model. Nothing is installed on your computer.

WHERE WE ARE

Lab 02 picked the tools. Lab 03 shows the thinking.

Lab 02 · Tool Use

The model chose a tool and read the result. It could reach the evidence.

But you could not see why it chose each call.

TODAY

Lab 03 · ReAct

Now the model states a thought before every action, then uses each observation to pick the next step.

The reasoning is on the page, one step at a time.

THE KEY IDEA

Three moves, over and over

Thought

The model says what it needs to know next, and why.

Action

It calls exactly one tool to get that one fact.

Observation

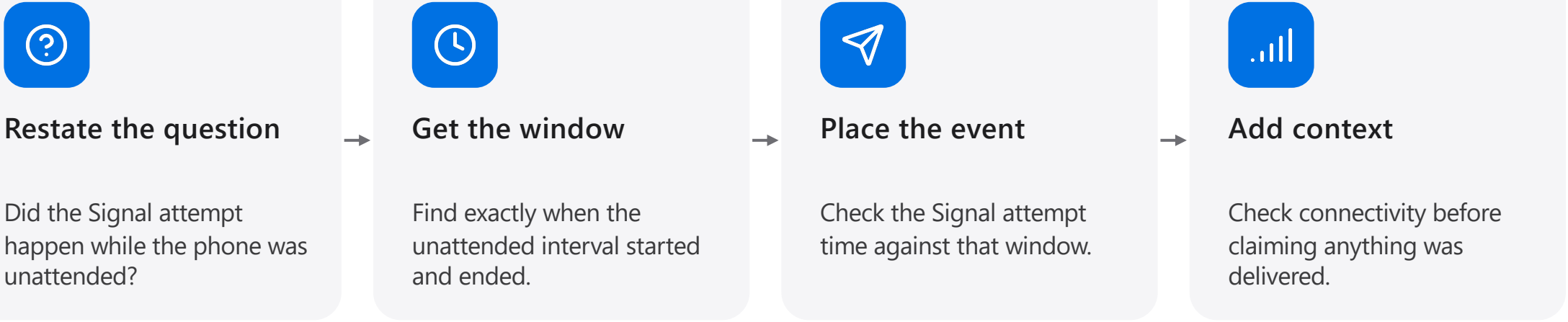
It reads the result, which shapes the next thought.

ONE TOOL PER TURN, ON PURPOSE

ReAct deliberately takes one step at a time. Each observation is allowed to change the plan, so the model is pushed to check an assumption before it charges ahead on it.

THE EXAMINER'S HABIT

A timeline, one check at a time



THIS IS ALREADY A REASON-ACT-OBSERVE LOOP

A careful examiner never answers a timing question in one leap. They line up one fact, see what it implies, then decide what to check next. ReAct teaches a model the same patience.

THE PROBLEM

The leap from attempt to conclusion

THE TEMPTING SHORTCUT

"There was a Signal event, so the image was sent during the unattended interval."

WHAT THAT LEAP SKIPS

Was it inside the window?

An event exists, but its time was never compared to the interval.

Was there connectivity?

Mobile data was offline. An attempt can log without a connection.

Attempt is not delivery

A send attempt and a delivered file are two different claims.

THE DISCIPLINE

Let each observation choose the next step

1	**Name the biggest unknown**	Before any tool, say what you most need to learn right now.
2	**Call one tool for it**	Not three. One, aimed at that single uncertainty.
3	**Read what it changed**	Let the result, not your hunch, decide what to check next.
4	**Answer only when ready**	Stop when the observations support the claim, and no sooner.

The loop is slow on purpose. Slow is how you stay inside the evidence.

THE GAP

A black-box answer cannot be checked

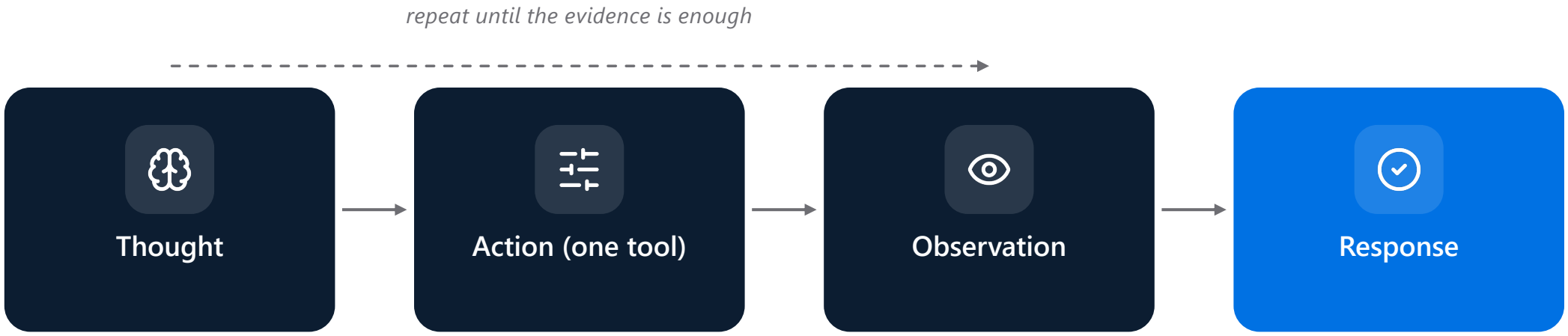


IF YOU CANNOT SEE THE STEPS, YOU CANNOT TRUST THE ANSWER

A model that returns only a conclusion gives you nothing to check. Did it compare the right timestamps? Did it notice the phone was offline? ReAct makes every step visible, so a reviewer can follow the reasoning and catch a wrong turn before it becomes a wrong verdict.

THE ANSWER

The ReAct loop



ONE STEP PER TURN

The model reasons, acts once, and observes. The loop is what makes the reasoning incremental and visible.

WHEN DOES IT STOP?

When the model can answer from what it has observed, it writes a final <response> instead of another thought.

WHAT MAKES THE LOOP WORK

Short-term memory

`chat_history`

- **Q** the question
- **T** every thought
- **A** every tool call
- **O** every observation

a running list, in order

Each turn can see the last

The model reads the whole trail, so turn 3 builds on turns 1 and 2.

You can read it too

The same list is the audit trail a reviewer follows after the fact.

Nothing hides outside it

There is no secret state. If it is not in the history, it did not happen.

THE INVESTIGATION

Case RA-2026-009

THE DEVICE

PHONE

Samsung Galaxy A54, Android 14

ACQUIRED

Cellebrite UFED, full file system

THE REPORT

phone left unattended briefly

THE APP

Signal, an encrypted messenger

THE QUESTION

During the unattended interval, did the phone try to send an image through Signal, and was it delivered?

A window

The phone was unattended from 14:10 to 14:25 UTC.

An attempt

A Signal image attempt is logged at 14:16 UTC, inside the window.

A boundary

Data was offline then, and only returned at 14:28, after the window.

YOUR INSTRUMENTS

Three tools, one slice each

get_incident_window

no inputs

Returns the start and end of the unattended interval.

get_message_attempt

app

Returns the Signal image-attachment attempt and its time.

get_network_restore_time

no inputs

Returns when mobile data came back online.

Each tool answers one question, so the loop has to reason its way from one to the next.

WATCH THE LOOP REASON

Three turns to a bounded answer

	*"I need the time window first."* `get_incident_window`	**-> 14:10 to 14:25 UTC**	Now later events have something to be compared against.
	*"Is the Signal attempt inside it?"* `get_message_attempt("Signal")`	**-> attempt at 14:16 UTC**	Inside the window. But data was offline at the time.
	*"Did connectivity ever return?"* `get_network_restore_time`	**-> restored 14:28 UTC**	After the window closed. An attempt, not a confirmed delivery.

Each thought came from the last observation. That is ReAct.

THE WHOLE POINT OF THE CASE

An attempt is not a delivery

What the evidence supports

- An image attempt through Signal was logged.
- It happened at 14:16 UTC, inside the window.
- The reasoning trail is complete and checkable.

What it does not establish

- That the image was delivered to anyone.
- Data was offline; it returned only at 14:28.
- Who was holding the phone at the time.

PRACTICAL

Your turn, in the browser

1	**Open + Part 0 setup**	Click the Colab badge. Steps 0.1 to 0.4: install, paste a free Groq key (hidden, not saved), write the case files, load the toolbox.
2	**Part A: turn by turn**	A.1 builds three tools, A.2 starts the memory. A.3 to A.5 run one turn each: Thought, Action, Observation. A.6 asks for the answer.
3	**Part B: ReactAgent**	B.1 lets the agent run the whole loop on its own, up to 10 rounds. Read every Thought, Tool call, and Tool result line.
4	**Your bounded answer**	Four parts: step log, the interval, the attempt time (inside: True or False), and a conclusion that stops at “attempt.”
5	**Compare the two runs**	Same three observations, in a sensible order? Did it avoid claiming delivery? Rewrite any claim that went too far.

Before every code cell, a short note tells you its Purpose, its Input, How it works, and its Output.

SectraTEK

Show your work, one step at a time.

Next lab: Planning, where the agent writes the whole plan first, then works the steps and replans when the evidence changes.

